COMP3702 Artificial Intelligence (Semester 2, 2025)
Assignment 3: Reinforcement Learning

Name: Tsz Fung Kung

Student ID: 48996066

Student email: s4899606@student.uq.edu.au

Note: Please edit the name, student ID number and student email to reflect your identity. When submitting to Gradescope, ensure you mark the start of each question.

Question 1 (Complete your full answer to Question 1 on the remainder of page 1)

```python
#import ploting library
import matplotlib.pyplot as plt
import numpy as np

# Store rewards per episode
all_rewards = []

# compute the r100 of the rewards per episode
def compute_r100(rewards):
    r100 = []
    for i in range(len(rewards)):
        start = max(0, i - 99)
        r100.append(np.mean(rewards[start:i+1])) #R100 [i] = mean(all_rewards[max (0, i-99) : i + 1])
    return r100

# plot the rewards and the r100
def plot_rewards_and_r100(rewards, env_name, net_name):
    r100 = compute_r100(rewards)
    plt.figure(figsize=(8,5))
    plt.plot(rewards, label="Reward", alpha=0.3)
    plt.plot(r100, label="R100 (moving average)", linewidth=2)
    plt.xlabel("Episode")
    plt.ylabel("Reward")
    plt.title(f"{env_name} {net_name}")
    plt.legend()
    plt.grid(True)
    plt.savefig(f"plots/{env_name}_{net_name}_R100.png", dpi=150)
```

1. Variable used to store episode rewards

A Python list named `all_rewards` is used to record the total reward obtained in each episode. Inside the main training loop of `dqn_gym.py`, after each episode finishes (when `is_done == True`), the accumulated reward (`episode_reward`) is appended to this list: **all_rewards.append(episode_reward)**

This list provides the raw data for both the **per-episode reward plot** and the **R100 moving average**. It grows dynamically as training progresses, with one entry per completed episode.

2. Where the plotting function is called

After the training loop completes (or periodically every 100 episodes), the plotting function plot_rewards_and_r100(all_rewards, args.env, args.network) is called to **generate and save** the performance figure.

Placing this function at the end of training ensures that all collected rewards are visualised together. The resulting plot file is saved automatically to a folder (e.g., /plots/) using the environment name and network type in its filename for traceability: **plt.savefig(f"plots/{env_name}_{net_name}_R100.png", dpi=150)**

This function can also be called at intermediate checkpoints to monitor progress during training.

3. How R100 is computed

The R100 value represents the average reward over the most recent 100 episodes, used to smooth short-term fluctuations and indicate long-term performance stability. For each episode *i*, the value is calculated as:

R100[i] = mean (all_rewards [max (0, i–99): i+1])

This logic ensures that for episodes **1–99**, where fewer than 100 episodes exist, a **prefix average** (mean of all available rewards so far) is used instead of skipping values. The resulting r100 list has the same length as all_rewards, allowing both to be plotted on the same x-axis.

To support later analysis and reproducibility, the all_rewards list and hyperparameters can be saved after training:

np.savez('plots/training_log.npz', rewards=np.array(all_rewards), **params)

This allows the data to be reloaded and plotted again without retraining.

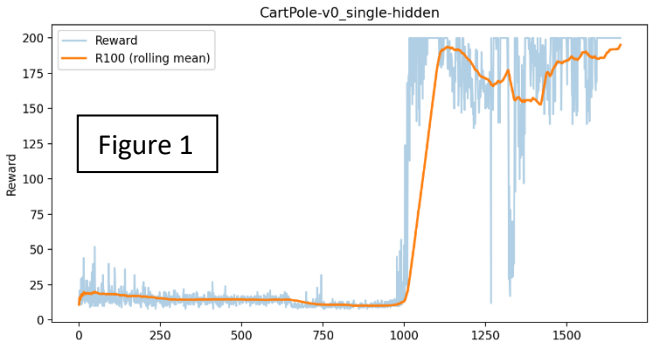
Question 2 (Complete your full answer to Question 2 on page 2)

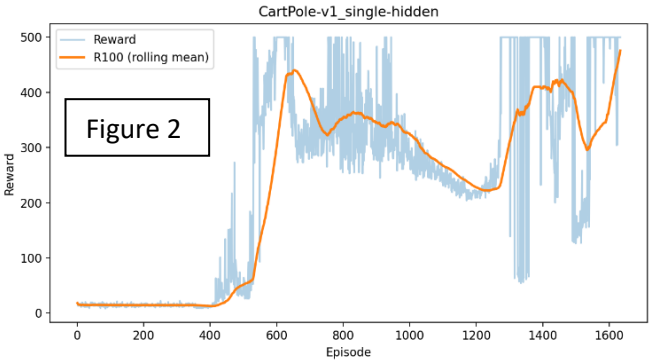
Figures 1 and 2 show the learning curves for CartPole-v0 and v1 using identical hyperparameters. To evaluate the performance of the DQN agent, two environments were tested using the same single-hidden-layer network and identical hyper-parameters. The difference lies only in episode length and termination conditions:

CartPole-v0: maximum episode length = 200 steps **CartPole-v1:** maximum episode length = 500 steps

Both environments reward +1 for every step the pole remains upright, so a higher R100 value (100-episode moving average reward) indicates better long-term control.

Observations from the R100 Plots





CartPole-v0 (Figure 1):

- The R100 curve remains near 10–20 during early training, showing the agent is exploring randomly.
- Around episode 1000, R100 rises sharply and stabilises near 195, meeting the success threshold (solved criterion ≥ 195).
- After convergence, fluctuations are minimal, showing the policy has become stable and robust.

CartPole-v1 (Figure 2):

- Learning begins more slowly, with R100 remaining below 50 until roughly episode 400–600.
- The average reward then increases to around 450–480 but fluctuates heavily over the remainder of training.
- Although the agent eventually achieves high rewards, the curve shows instability and slower convergence.

Analysis

Factor	CartPole-v0	CartPole-v1	Explanation
Episode length	200	500	Longer horizon makes v1 harder – agent must maintain balance for more steps
Convergence speed	≈ 1000 episodes	≈ 1500–1600 episodes	v0 converges faster due to shorter sequences and smaller cumulative error
Reward stability	High	Medium	v1 shows larger variance from exploration noise
Final R100	≈ 195	≈ 480	v1 achieves higher ceiling but with more fluctuation

Several factors explain the observed difference:

1. Task complexity: The longer horizon in v1 increases temporal credit assignment difficulty—small control errors accumulate over time, making balance harder to maintain.
2. Exploration variance: Longer episodes generate higher return variance; ε-greedy exploration causes greater oscillations in the R100 curve.
3. Learning stability: DQN updates converge faster in v0 due to shorter episode lengths and fewer cumulative prediction errors.

Overall, the R100 plots demonstrate that both environments are solvable by the DQN, but CartPole-v0 **learns faster and more stably**, whereas CartPole-v1 **achieves higher final rewards with slower and more unstable convergence** due to its extended episode duration and higher reward horizon. These results confirm that environment design—specifically episode length and reward horizon—strongly affects DQN learning stability.

Question 3 (Complete your full answer to Question 3 on page 3)

(a) Two key similarities

1. Both methods are **based on the Bellman optimality equation**, updating value estimates toward the expected discounted future reward.
2. Both aim to compute the **optimal policy** by iteratively improving state or action-value estimates ($Q^*(s, a)$ or $V^*(s)$).
3. Both employ **bootstrapping**, using current estimates of future values rather than waiting for full episode outcomes.
4. Both are **iterative dynamic programming–style algorithms** that converge toward the optimal value function through repeated backups.

(b) One key difference

Value Iteration is a **model-based** dynamic programming algorithm that requires full knowledge of the **transition probabilities** $P(s' \mid s, a)$ and **reward function** $R(s, a, s')$, updating **all states synchronously** in each iteration.

In contrast, **Q-learning** is a **model-free** reinforcement learning method that learns optimal Q-values **directly from sampled experience** (s, a, r, s') and updates **only the visited states asynchronously** using the **temporal-difference (TD) error**.

Question 4 (Complete your full answer to Question 4 on page 4)

(a) Loss function in DQN

In DQN, the loss is based on the **temporal-difference (TD) error** between the target and the predicted Q-values:

$$L(\theta) = (y_t - Q(s_t, a_t; \theta))^2, y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a'; \theta^-)$$

where $y_t$is the **target value** (computed using the target network) and $Q(s_t, a_t; \theta)$is the **current network's prediction**. Minimizing this loss via gradient descent reduces the TD error between the estimated and true returns. This loss corresponds to the mean squared TD error minimized via backpropagation.

(b) Variables in code

From dqn_gym.py:

```
state_action_values = Q_s.gather(1, actions_v.type(torch.int64).unsqueeze(-1)).squeeze(-1)
next_state_values = target_net.forward(next_states_v).max(1)[0]
expected_state_action_values = rewards_v + next_state_values.detach() * params['gamma'] * (1 - dones_v)
loss = F.mse_loss(state_action_values, expected_state_action_values)
```

Target (y_t): expected_state_action_values

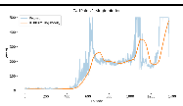
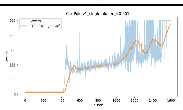
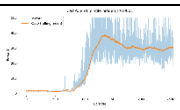
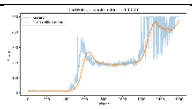
Prediction: state_action_values

Loss: F.mse_loss()

(c) Why use a separate target network

If the same network were used for both prediction and target computation, both sides of the TD error would shift simultaneously, causing **training instability or divergence**. A **separate target network** Q_{target} provides a fixed reference, smoothing learning and preventing feedback loops between rapidly changing values.

(d) Effect of target update and learning parameters (Detail figures see appendix)

Experiment	Configuration	Observation
Figure 1 – Hard update (baseline) 	Periodic targets sync every 1000 steps	R100 rises to ≈480 but shows sharp oscillations after each hard sync.
Figure 2 – Soft update (τ = 0.005) 	Gradual parameter blending	R100 curve is smoother and converges steadily to ~500 with reduced variance.
Figure 3 – Lower learning rate (lr = 0.0001) 	Smaller gradient steps	Learning is slower; R100 peaks near 300 then plateaus, indicating under-learning.
Figure 4 – Faster ε-decay (ed = 20000) 	Exploration reduced too early	Two-phase curve: slow initial growth, late recovery after 1200 episodes.

Analysis:

The baseline (hard sync) produces abrupt TD-error jumps when the target network updates suddenly.

Soft updates maintain smoother convergence because parameters evolve continuously.

A too-small learning rate prevents sufficient weight adjustment, while overly rapid ε-decay limits exploration early and delays convergence.

Overall, soft target updates (τ = 0.005) achieve the most stable and consistent R100 performance (~480–500),

confirming that gradual target tracking and balanced exploration promote more reliable DQN learning.

Question 5 (Complete your full answer to Question 5 on page 5)

(a) R100 plots for different learning rates

Using the **CartPole-v1** environment, DQN agents were trained with **three fixed learning-rate values** while keeping other parameters constant.

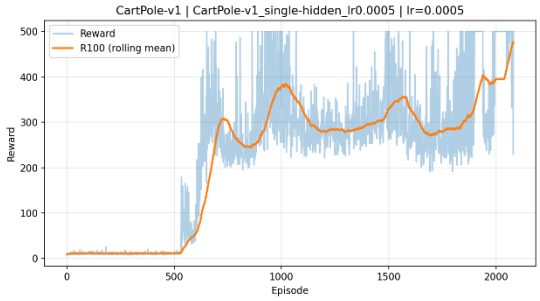


Figure 5.1

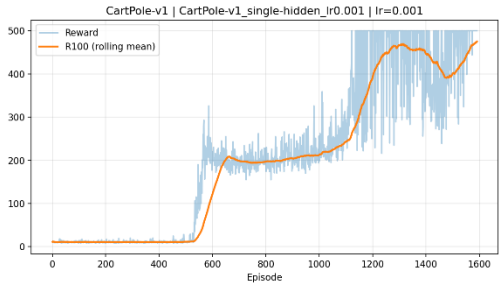


Figure 5.2

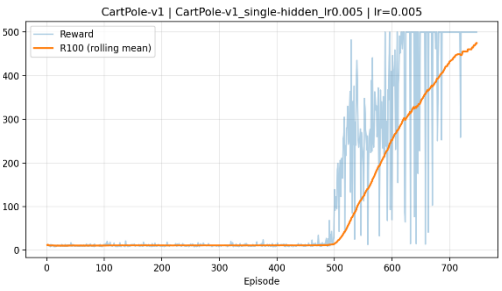


Figure 5.3

learning-rate = 0.0005 (Figure 5.1) – slow, smooth improvement; converges gradually after ≈ 1500–2000 episodes.

learning-rate = 0.001 (Figure 5.2) – faster convergence and stable plateau around R100 ≈ 450 by ≈ 1500 episodes.

learning-rate = 0.005 (Figure 5.3) – very rapid rise (≈ 700 episodes) but stronger fluctuations.

Across all runs, R100 eventually approaches the task maximum (≈ 500), but the **speed and stability** differ markedly.

(b) Effect of varying learning rate

Observation:

Higher lr → faster early learning: The 0.005 run achieves high R100 within ~700 episodes, while 0.001 takes ≈ 1200–1500 episodes, and 0.0005 learns much more slowly.

Trade-off between speed and stability: The 0.005 curve oscillates more, indicating unstable updates; the 0.0005 curve is smoother but converges slowly; 0.001 shows a good balance.

Final performance similar: All rates can eventually reach near-optimal R100 if trained long enough, but their transient behaviours differ significantly.

Interpretation

A small learning rate makes TD updates conservative → slow but stable learning.

A moderate rate (0.001) gives the best trade-off between convergence speed and stability.

A large rate accelerates learning but risks overshooting and oscillations, as the Q-values fluctuate before stabilizing.

Even with DQN's stabilizing mechanisms, too large lr still increases variance and may hinder convergence. The learning rate strongly influences both convergence speed and stability of DQN training.

(c) Effect of an Excessively High Learning Rate

When lr is too high, each gradient step over-adjusts the Q-network's parameters, causing the target and online networks to chase rapidly changing Q-value estimates. This leads to:

- **Oscillations** in the estimated Q-values and policy behavior,
- **Unstable R100 curves** and inconsistent episode rewards,
- Potential **divergence** if weight updates overshoot the optimum repeatedly.

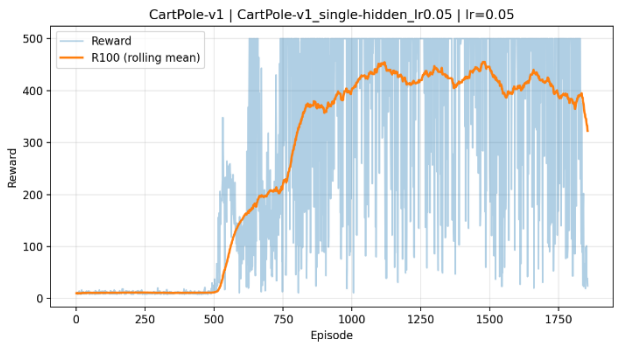


Figure 5.4 – with a lr = 0.05

Question 6 (Complete your full answer to Question 6 on page 6)

(a) Equation & Purpose of ε-Greedy Exploration

Epsilon is used in the ε-greedy action-selection rule that controls the balance between exploration and exploitation. In Deep Q-Networks (DQN), the **epsilon-greedy strategy** balances **exploration** (trying new actions) and **exploitation** (choosing the best-known action). The probability of selecting a random action at time $t$is controlled by the exploration parameter ε_t, which typically decays exponentially over training steps:

$$\boxed{\varepsilon_t = \varepsilon_{\text{final}} + (\varepsilon_{\text{start}} - \varepsilon_{\text{final}})\exp\left(-\frac{t}{\varepsilon_{\text{decay}}}\right)}$$

At each environment step t:

- With probability ε_t: select a **random action** (→ exploration).
- With probability $1 - \varepsilon_t$: select the **greedy action** that maximizes $Q(s_t, a)$(→ exploitation).

Purpose:

- Early in training, $\varepsilon_t$is close to 1 so the agent explores widely, collecting diverse experience that improves generalisation.
- As learning progresses, $\varepsilon_t$decays towards $\varepsilon_{\text{final}}$, reducing randomness and allowing the policy to exploit learned value estimates.
- Retaining a small non-zero $\varepsilon_{\text{final}}$keeps occasional exploration to prevent over-fitting to sub-optimal policies.

This mechanism directly affects how efficiently an agent discovers good behaviors. Too much exploration slows convergence, whereas too little can trap the policy in local optima.

(b) Effect of ε_Start, ε_Final, and ε_Decay on Training Performance

Parameter	Description	Effect on Learning Behaviour
$\varepsilon_{\text{start}}$	Initial exploration level (often 1.0).	Controls how randomly the agent behaves at the start. A higher value increases diversity of early experience but delays exploitation of good actions.
$\varepsilon_{\text{final}}$	Minimum exploration value (e.g. 0.001–0.05).	Ensuring the agent never becomes completely deterministic. A small ε_final reduces long-term randomness but may be over-exploited; a larger ε_final keeps exploration but can cause noisy performance.
$\varepsilon_{\text{decay}}$	Controls the rate of exponential decay.	A smaller decay constant causes ε to drop quickly (faster exploitation, risk of premature convergence). A larger decay maintains exploration longer, stabilizing learning but requiring more episodes to converge.

Interpretation:

- **ε_decay = 1000** → rapid decay; the agent becomes greedy almost immediately, risking inadequate exploration and poorer final performance.
- **ε_decay = 5000** → moderate decay; common default providing balanced exploration early and efficient exploitation later.
- **ε_decay = 20000 or 40000** → slow decay; the agent remains exploration for longer, gathering more diverse experiences at the cost of slower convergence.

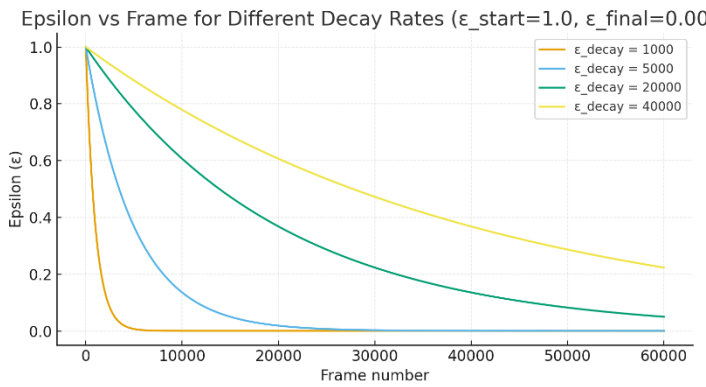


Figure 6: Epsilon decay schedules in DQN for different ε decay values.

Question 7 (Complete your full answer to Question 7 on page 7, and optionally page 8)

Environment and Success Criterion

I applied DQN to **LunarLander-v3** (Gymnasium). This environment requires a lander to stabilise and land softly between two flags. A policy is considered **solved** when the **100-episode moving-average reward (R100)** exceeds **+200** and remains stable. This corresponds to sustained R100 ≥ 200 with stable, smooth landings between the flags without crashes or excessive fuel use. While the threshold corresponds to consistent controlled landings without crashes or excessive fuel use.

Selected Hyperparameters and Rationale

We experimented with **three core hyperparameters** (plus one optional exploration factor) to study their effect on learning efficiency and stability. All other hyperparameters were kept constant across experiments so that only the stated hyperparameter differs.

Hyperparameter	Values Tested	Rationale
Network architecture	single-hidden / two-hidden / duelling-DQN	Tests representational power vs. sample efficiency and training stability.
Learning rate (α)	0.0005 / 0.001 / 0.005	Controls the step size in gradient descent. Too small → slow learning; too large → unstable updates.
Target-network sync interval	500 / 1000 / 5000 steps	Determines how often target parameters are updated. Frequent → noisy; infrequent → stale targets.
Epsilon-decay (ε_decay)	2000 / 5000 / 10000	Controls exploration schedule; faster decay → premature exploitation.

Experimental Results (R100 vs Episodes)

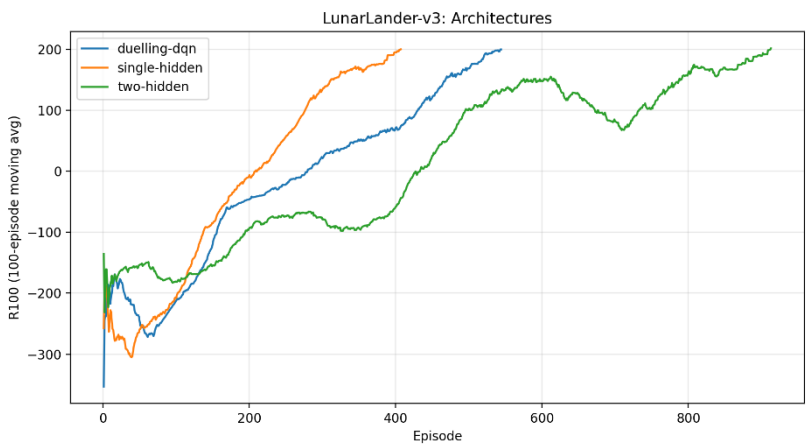


Figure 7(a) – Architectures

- **single-hidden** learns fastest and first reaches R100 ≈ 200.
- **duelling-DQN** converges slightly slower but produces smoother mid-training curves.
- **two-hidden** lags, likely due to greater parameter complexity with limited data.

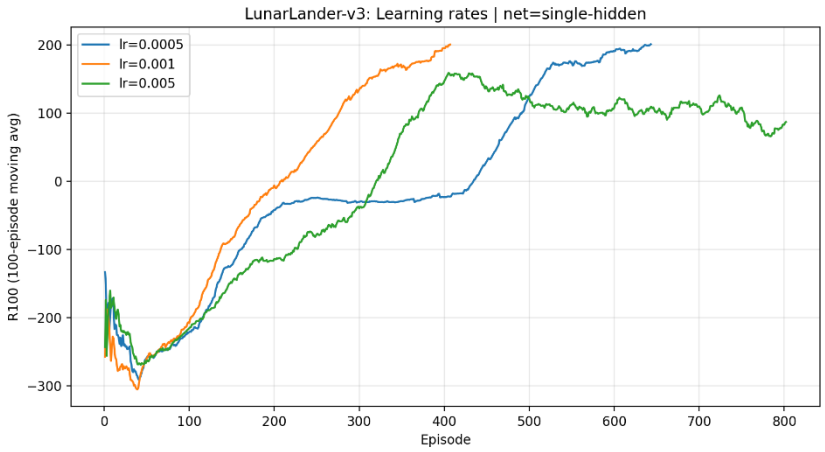


Figure 7(b) – Learning Rate

- **α = 0.001** achieves the best balance of speed and stability, rising smoothly to ≈ 200.
- **α = 0.0005** is stable but slower.
- **α = 0.005** improves quickly early on but oscillates and plateaus ≈ 100–150.

Question 7 (continue Question 7 here if needed, otherwise leave it blank)

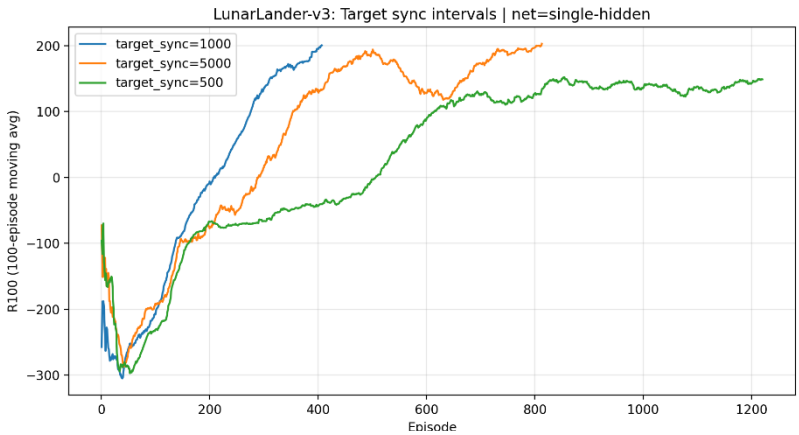


Figure 7(c) – Target-Network Sync Interval

- **1000 steps:** best performance; fast, smooth convergence to ≈ 200.
- **500 steps:** too frequent updates; increased variance.
- **5000 steps:** updates too stale; slower improvement.

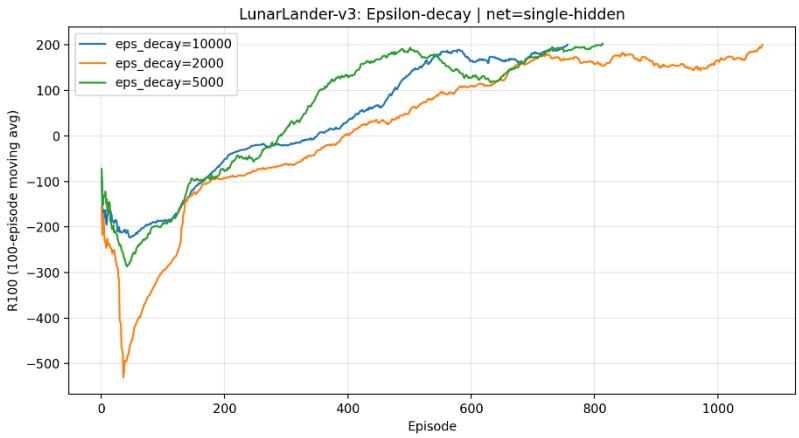


Figure 7(d) – Epsilon-Decay

- **ε_decay = 5000** strikes a good balance, escaping early negative rewards and converging steadily.
- **ε_decay = 2000** decays exploration too rapidly → local minima.
- **ε_decay = 10000** maintains exploration longer but slows exploitation.

Adjustments and Observations

- Starting from CartPole defaults, increasing environment complexity (continuous state, denser rewards) required **smaller learning rates** and **moderate target-sync intervals** for stable convergence.
- Early tests with high α or low sync (500) produced noisy value estimates; adjusting to α = 0.001 and sync = 1000 yielded smoother learning.
- Exploration tuning showed that moderate decay (5000) helps the agent escape poor local behaviors without over-randomness later.
- Dueling DQN added representational power but brought minimal extra benefit compared with the simpler single-hidden network given the same training time.

Final Hyperparameter Settings

Parameter	Final Value
Network architecture	single-hidden
Learning rate (α)	0.001
Target-network sync interval	1000 steps
ε schedule	ε_start = 1.0, ε_final = 0.001, ε_decay = 5000

Demonstration of Solved Performance

With these final settings, the agent achieved **R100 ≈ 200** around **600–800 episodes**, satisfying the success criterion. Qualitatively, the trained policy stabilises mid-air, reduces lateral velocity before touchdown, and lands between the flags with minimal bounce—clear evidence of a **successful DQN solution** for LunarLander-v3.

The experimental outcomes align closely with theoretical expectations from DQN literature. Moderate learning rates and target-sync intervals yield the most stable value-function updates, while balanced exploration (ε_decay ≈ 5000) ensures robust convergence in a continuous control task like LunarLander.

Appendix/References (Include references and usage of Generative AI on the last page)

References

Bialkowski, A. (2025). *COMP3702 Artificial Intelligence – Modules 3–5 Lecture Notes* (Weeks 7–12). The University of Queensland, School of Electrical Engineering and Computer Science.

Farama Foundation. (2024). *Gymnasium Documentation: CartPole-v1 & LunarLander-v3 environments*. Retrieved from https://gymnasium.farama.org

PyTorch. (2024). *Reinforcement Q-learning Tutorial*. Retrieved from https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Retrieved from http://incompleteideas.net/book/RLbook2020.pdf

UQ COMP3702 GitHub. (2025). *COMP3702 DQN Example Code and Tutorials* [Course Repository]. Retrieved from https://github.com/comp3702

Acknowledgement of Generative AI Tools

Tool / Platform	Use	Prompt(s) / Description	Section(s) Assisted	Date
ChatGPT (GPT-5, OpenAI)	Explanation of DQN concepts, TD-loss derivation, and epsilon-greedy formula	"Explain DQN loss function and identify code variables for target and prediction in dqn_gym.py"	Q4 (a–b)	2025-10-29
ChatGPT (GPT-5, OpenAI)	Assisted with report clarity and consistency (rewriting sections for concise academic tone)	"Polish Q3 similarities/differences between Q-learning and Value Iteration"	Q3	2025-10-29
ChatGPT (GPT-5, OpenAI)	Formatting and construction of Appendix and Reference section according to APA 7th style	"Create Appendix/References (include references and usage of Generative AI on the last page)"	Appendix	2025-10-30

Notes on AI Usage

All AI-generated text was reviewed, edited, and verified by the student. The model was used as a writing and explanation aid; all data analysis, code execution, and interpretation of experimental results were performed independently by the author.

Figure References (Question 4)

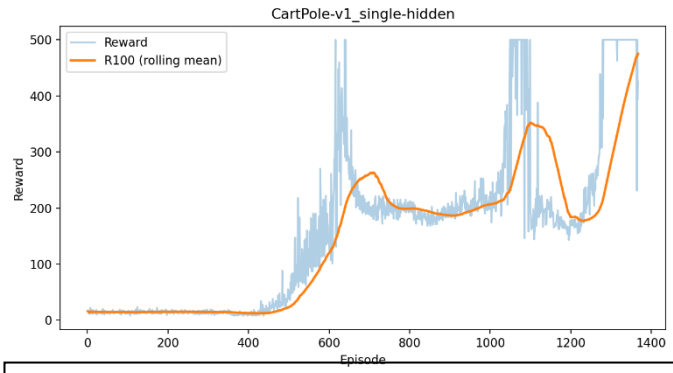


Figure 1. CartPole-v1 (single-hidden, hard sync = 1000)

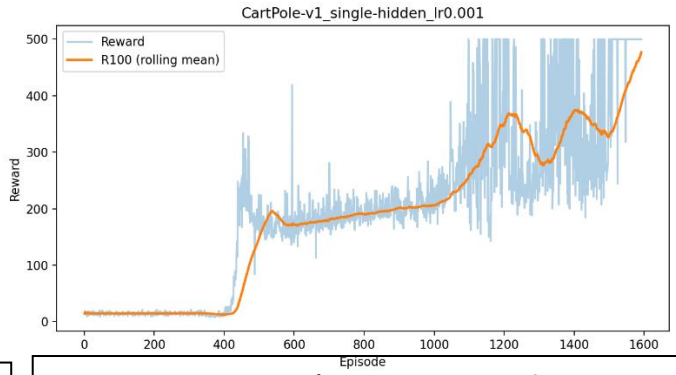


Figure 2. CartPole-v1 (single-hidden, soft update τ = 0.005, lr = 0.001)

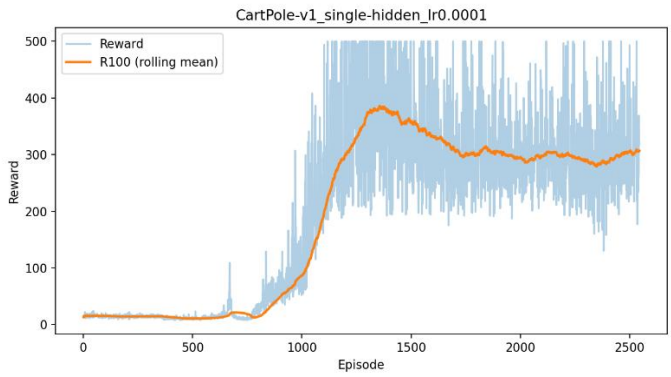


Figure 3. CartPole-v1 (single-hidden, lr = 0.0001)

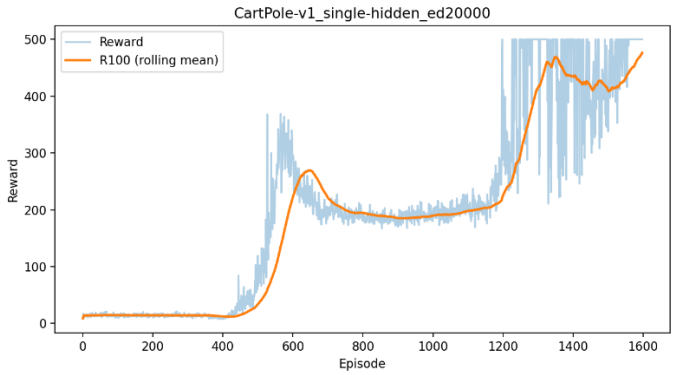


Figure 4. CartPole-v1 (single-hidden, ε-decay = 20000)